ZAP by Checkmarx Scanning Report on DVWA

Report Overview

The ZAP by Checkmarx scanning report was generated on January 3, 2026, using ZAP version 2.17.0. It scans the site http://testphp.vulnweb.com, a known vulnerable test application. The report includes all contexts and sites by default, covering risk levels from High to Informational and confidence levels from User Confirmed to Low (excluding False Positives).

Key summaries:

- Total unique alert types: 12
- Total alert instances: 40 (calculated from per-type counts)
- Alert distribution by risk (based on types): High (1 type, 8.3%), Medium (3 types, 25.0%), Low (4 types, 33.3%), Informational (4 types, 33.3%)
- Alert distribution by confidence (based on types): High (4 types, 33.3%), Medium (4 types, 33.3%), Low (4 types, 33.3%)

Key Findings: Vulnerabilities (Alerts)

The scan identified the following alert types, grouped by risk level. Each includes the risk rating, number of instances found, and percentage of total types. These represent security issues detected on the site.

Alert Type	Risk	Instances	Percentage
Cross Site Scripting (DOM Based)	High	1	8.3%
Absence of Anti-CSRF Tokens	Medium	4	33.3%
Content Security Policy (CSP) Header Not Set	Medium	4	33.3%
Missing Anti-clickjacking Header	Medium	4	33.3%
In Page Banner Information Leak	Low	3	25.0%
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	4	33.3%
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	4	33.3%
X-Content-Type-Options Header Missing	Low	4	33.3%
Authentication Request Identified	Informational	1	8.3%

Charset Mismatch (Header Versus Meta Content-Type Charset)	Informational	4	33.3%
Modern Web Application	Informational	5	41.7%
User Controllable HTML Element Attribute (Potential XSS)	Informational	2	16.7%

High Severity Findings

Title	Description	Severity	Impact	Remediation	Instances
Cross Site Scripting (DOM Based)	DOM-based XSS occurs when client-side scripts manipulate the DOM using untrusted input, potentially allowing attackers to execute malicious code in the user's browser.	High	Attackers can steal session cookies (unless protected by HttpOnly), execute arbitrary code, or compromise user data.	Use vetted libraries or frameworks like Microsoft's Anti-XSS library or OWASP ESAPI to properly encode output and avoid this weakness.	**1**

Medium Severity Findings

Title	Description	Severity	Impact	Remediation	Instances
Absence of Anti-CSRF Tokens	No anti-CSRF tokens were found in HTML submission forms, which can allow cross-site request forgery attacks where victims unknowingly	Medium	Attackers can exploit trust to force unauthorized actions like fund transfers or data changes, especially if combined with XSS.	Use a vetted library like OWASP CSRFGuard; ensure the application is free of XSS vulnerabilities as they can bypass CSRF defenses.	**4**

	perform actions on a site.				
Content Security Policy (CSP) Header Not Set	Content Security Policy (CSP) is missing, which helps mitigate attacks by specifying approved content sources for browsers to load.	Medium	Increases risk of XSS, data injection, theft, site defacement, or malware distribution.	Implement CSP HTTP headers to declare allowed sources for JavaScript, CSS, images, etc.; refer to OWASP CSP Cheat Sheet for configuration.	**4**
Missing Anti-clickjacking Header	The response lacks protection against clickjacking, such as X-Frame-Options or CSP frame-ancestors directives.	Medium	Attackers can overlay malicious iframes, tricking users into clicking hidden elements to perform unintended actions.	Add X-Frame-Options : DENY or SAMEORIGIN, or use CSP with frame-ancestors 'self'.	**4**

Low Severity Findings

Title	Description	Severity	Impact	Remediation	Instances
In Page Banner Information Leak	The page contains banners or text leaking server/software information, such as version details.	Low	Reveals tech stack, allowing attackers to target known vulnerabilities in specific versions.	Minimal direct impact but aids reconnaissance for targeted attacks.	Configure the server to suppress banners; for Apache, use ServerSignature Off and ServerTokens Prod; for Tomcat, customize error pages and

					server directive.
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	The X-Powered-By header exposes underlying technology (e.g., PHP version).	Low	Helps attackers identify exploitable components or versions.	Remove or set the X-Powered-By header to a generic value in server configuration (e.g., in PHP.ini or web server settings).	**4**
Server Leaks Version Information via "Server" HTTP Response Header Field	The Server header includes version details (e.g., Apache/2.4.1).	Low	Similar to above, enables version-specific attacks.	Suppress version info; for Apache, set ServerTokens Prod; for Nginx, server_tokens off.	**4**
X-Content-Type-Options Header Missing	The anti-MIME-sniffing header is absent, allowing browsers to guess content types.	Can lead to misinterpreted content, potentially enabling XSS or other injection attacks in older browsers.	Set X-Content-Type-Options: nosniff in HTTP responses.	4	

Informational Severity Findings

Title	Description	Severity	Impact	Remediation	Instances
Authentication Request Identified	ZAP detected requests that appear to handle authentication, such as login forms.	Informational	No direct security impact; aids in identifying auth mechanisms for further review.	N/A; consider automating auth handling in ZAP for deeper scans.	**1**
Charset Mismatch (Header Versus Meta Content-Type Charset)	Mismatch between HTTP Content-Type charset and HTML meta tag charset.	Informational	May cause rendering issues or minor security risks in edge cases (e.g., encoding attacks).	Ensure consistent charset declaration (e.g., UTF-8) in both header and meta tags.	**4**
Modern Web Application	The application appears to be a modern web app, likely relying on JavaScript or dynamic content.	Informational	No impact; indicates need for tools like Ajax Spider for full coverage.	N/A; use ZAP's Ajax Spider for JS-heavy apps.	**5**
User Controllable HTML Element Attribute (Potential XSS)	User input from queries or POST data controls HTML attributes, creating XSS hotspots.	Informational	Potential for XSS if not sanitized, leading to code injection.	Manually review and sanitize inputs; use encoding per OWASP XSS Prevention Cheat Sheet.	

Insights and Statistics

The report includes additional insights on the scan process and site behavior:

- ZAP errors logged: 7 (Low warning; check zap.log for details)
- ZAP warnings logged: 23 (Low warning; check zap.log for details)
- Network failures: 39%
- Responses with 2xx status: 94%
- Responses with 3xx status: 1%
- Responses with 4xx status: 3%
- Endpoints by content type: application/x-shockwave-flash (2%), image/gif (4%), image/jpeg (10%), image/x-icon (2%), text/css (6%), text/html (75%)
- Endpoints by method: GET (89%), POST (10%)
- Total endpoints: 49
- Slow responses: 100%

These findings indicate significant security gaps, particularly in cross-site scripting protection, CSRF prevention, and HTTP header configurations, which are common in test environments like this. High-risk issues like DOM-based XSS could allow attackers to execute malicious scripts in users' browsers. Recommendations would typically involve implementing missing headers, tokens, and policies as per standard web security practices.